

Phase 8: Binary Operations, Merge, and Set Operations in Array ADT

1. Big Idea of Phase 8

Phase 8 teaches operations that work with **two arrays**.

In earlier phases, most operations worked on one array:

```
display(arr);  
reverse(&arr);  
insertSort(&arr, 12);
```

In Phase 8, the operations usually take **two arrays** and create a **third result array**:

```
merge(arr1, arr2);  
Union(arr1, arr2);  
Intersection(arr1, arr2);  
Difference(arr1, arr2);
```

Simple idea:

Phase 8 = take two arrays and produce a useful third array.

The main topics in Phase 8 are:

1. Binary operations on arrays
2. Merging two sorted arrays
3. Set operations
4. Union
5. Intersection
6. Difference
7. Membership
8. Complexity of sorted and unsorted set operations
9. Common mistakes with result arrays and pointer movement

2. What is a Binary Operation?

A **binary operation** means an operation that uses **two inputs**.

In Array ADT, that usually means two arrays.

Example:

```
Array 1 = [2, 6, 10]
Array 2 = [3, 4, 7]
```

A binary operation may combine, compare, or process both arrays.

Examples of binary operations:

Operation	Meaning
Merge	Combine two sorted arrays into one sorted array
Union	Get all unique elements from both arrays
Intersection	Get only common elements
Difference	Get elements from one array that are not in the other
Membership	Check whether a value exists in a set/array

3. Array ADT Structure Used in Phase 8

We continue using the same Array ADT structure:

```
#define MAX 50

struct Array {
    int A[MAX];
    int size;
    int length;
};
```

Meaning:

Part	Meaning
<code>A</code>	The actual array storage
<code>size</code>	Total capacity of the array
<code>length</code>	Number of valid elements currently stored

Example:

```
struct Array arr = {{2, 6, 10, 15, 25}, 50, 5};
```

This means:

```
A = [2, 6, 10, 15, 25, _, _, ...]  
size = 50  
length = 5
```

Only these indexes are valid:

```
0, 1, 2, 3, 4
```

Important rule:

```
Use length when processing actual data.  
Use size only when checking capacity.
```

Part A: Merge Two Sorted Arrays

4. What Merge Means

Merge means combining two sorted arrays into one sorted array.

Example:

```
A = [2, 6, 10, 15, 25]  
B = [3, 4, 7, 18, 20]
```

Merged result:

```
C = [2, 3, 4, 6, 7, 10, 15, 18, 20, 25]
```

Simple idea:

Merge = combine two sorted arrays while keeping the final result sorted.

5. Important Rule: Both Arrays Must Be Sorted

Merge works correctly only if both input arrays are already sorted.

Example of sorted arrays:

```
A = [2, 6, 10, 15, 25]
B = [3, 4, 7, 18, 20]
```

These are sorted because values go from smaller to larger.

If arrays are not sorted, merge logic will fail.

Example of unsorted input:

```
A = [10, 2, 25, 6]
B = [4, 20, 3]
```

If we try to merge these directly, the result may not be sorted.

Important memory:

Merge assumes sorted input.
If input is not sorted, sort it first or use a different method.

6. Why Merge is Better Than Append + Sort

One simple but inefficient idea is:

1. Copy all elements of **A** into **C**.
2. Copy all elements of **B** into **C**.
3. Sort **C**.

But merge is better because both arrays are already sorted.

Merge does not need to sort again.

It only compares the current values and copies the smaller one.

Simple idea:

Because both arrays are already sorted, merge can build the final sorted array directly.

7. Three-Pointer Idea in Merge

Merge uses three index variables:

Pointer	Meaning
<code>i</code>	Current index in first array
<code>j</code>	Current index in second array
<code>k</code>	Current index in result array

Example:

```
A = [3, 8, 16, 20, 25]
B = [4, 10, 12, 22, 23]
C = []
```

Start:

```
i = 0
j = 0
k = 0
```

Meaning:

```
i points to A[0]
j points to B[0]
k points to C[0]
```

Compare:

A[i] and B[j]

The smaller value goes into:

C[k]

Then the pointer of the copied value moves forward.

8. Merge Main Logic

While both arrays still have elements:

Compare A[i] and B[j]

There are two main cases:

Case 1: A[i] is smaller

A[i] < B[j]

Copy A[i] into C[k] .

Then move:

i++
k++

Case 2: B[j] is smaller or equal

B[j] <= A[i]

Copy B[j] into C[k] .

Then move:

```
j++  
k++
```

After one array finishes, copy the remaining elements from the other array.

9. Merge Dry Run

Given:

```
A = [3, 8, 16, 20, 25]  
B = [4, 10, 12, 22, 23]
```

Initial values:

```
i = 0  
j = 0  
k = 0  
C = []
```

Dry run:

Step	Compare	Smaller value copied	Result C
1	3 vs 4	3	[3]
2	8 vs 4	4	[3, 4]
3	8 vs 10	8	[3, 4, 8]
4	16 vs 10	10	[3, 4, 8, 10]
5	16 vs 12	12	[3, 4, 8, 10, 12]
6	16 vs 22	16	[3, 4, 8, 10, 12, 16]
7	20 vs 22	20	[3, 4, 8, 10, 12, 16, 20]
8	25 vs 22	22	[3, 4, 8, 10, 12, 16, 20, 22]
9	25 vs 23	23	[3, 4, 8, 10, 12, 16, 20, 22, 23]
10	B finished	25	[3, 4, 8, 10, 12, 16, 20, 22, 23, 25]

Final result:

```
C = [3, 4, 8, 10, 12, 16, 20, 22, 23, 25]
```

10. Why We Need Leftover Loops in Merge

The main merge loop runs only while both arrays still have values:

```
while (i < arr1->length && j < arr2->length)
```

This means the loop stops when either array is finished.

But the other array may still have values left.

Example:

```
A = [2, 5, 8]
B = [1, 3, 4, 6, 7, 9]
```

A may finish first, but **B** still has elements.

So we need leftover loops:

```
while (i < arr1->length) {
    arr3->A[k++] = arr1->A[i++];
}

while (j < arr2->length) {
    arr3->A[k++] = arr2->A[j++];
}
```

Important point:

```
Only one leftover loop will actually copy values.
```

Why?

Because when the main loop stops, at least one array is already finished.

11. Merge Code

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 50

struct Array {
    int A[MAX];
    int size;
    int length;
};

void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}

struct Array* merge(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;

    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else {
            arr3->A[k++] = arr2->A[j++];
        }
    }

    while (i < arr1->length) {
        arr3->A[k++] = arr1->A[i++];
    }

    while (j < arr2->length) {
        arr3->A[k++] = arr2->A[j++];
    }

    arr3->length = k;
    arr3->size = MAX;

    return arr3;
}
```

```

int main() {
    struct Array arr1 = {{2, 6, 10, 15, 25}, MAX, 5};
    struct Array arr2 = {{3, 4, 7, 18, 20}, MAX, 5};

    struct Array *arr3 = merge(&arr1, &arr2);

    display(*arr3);

    free(arr3);

    return 0;
}

```

Expected output:

```

2 3 4 6 7 10 15 18 20 25

```

12. Explanation of Merge Code

Function header

```

struct Array* merge(struct Array *arr1, struct Array *arr2)

```

This means:

- The function receives the addresses of two arrays.
- It returns a pointer to a new result array.

Create indexes

```

int i = 0, j = 0, k = 0;

```

Meaning:

- `i` moves through `arr1`.
- `j` moves through `arr2`.
- `k` moves through `arr3`.

Create result array

```
struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));
```

This creates the result array in heap memory.

Main merge loop

```
while (i < arr1->length && j < arr2->length)
```

This loop continues while both arrays still have valid elements.

Copy smaller value

```
if (arr1->A[i] < arr2->A[j]) {  
    arr3->A[k++] = arr1->A[i++];  
} else {  
    arr3->A[k++] = arr2->A[j++];  
}
```

This copies the smaller current value into the result array.

Set final length

```
arr3->length = k;
```

This is important.

`k` tells us how many values were inserted into the result array.

Set result size

```
arr3->size = MAX;
```

The result array has capacity `MAX`.

Return result

```
return arr3;
```

The function returns the new merged array.

13. Important Memory Point: Why Use malloc?

Inside `merge`, the result array is created like this:

```
struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));
```

This creates `arr3` in heap memory.

That is useful because heap memory still exists after the function ends.

Wrong idea:

```
struct Array arr3;  
return &arr3;
```

This is unsafe because `arr3` is a local variable.

After the function ends, local variables are destroyed.

So returning the address of a local variable is dangerous.

Correct idea:

```
struct Array *arr3 = malloc(sizeof(struct Array));  
return arr3;
```

Simple memory:

```
Do not return the address of a local stack variable.  
Use heap memory or pass the result array from outside.
```

14. Merge Complexity

Let:

```
m = number of elements in arr1
n = number of elements in arr2
```

Merge copies every element exactly once.

So:

Complexity type	Result
Time complexity	$O(m + n)$
Space complexity	$O(m + n)$

Why time is $O(m + n)$:

Every element from both arrays is visited and copied once.

Why space is $O(m + n)$:

The result array stores all elements from both arrays.

Sometimes this is written as:

$\Theta(m + n)$

Theta means the bound is tight: merge really does proportional work to the total number of elements.

Part B: Set Operations

15. What is a Set?

A **set** is a collection of values where repeated values are usually not allowed.

Example:

{2, 4, 6, 8}

This is a set.

But this is usually not treated as a proper set:

```
{2, 4, 4, 6, 8}
```

Why?

Because `4` is repeated.

Important rule for Phase 8:

```
Set operations usually assume unique elements.
```

16. Set Operations in Phase 8

Phase 8 covers these set operations:

Operation	Meaning
Union	All unique elements from both arrays
Intersection	Only elements common to both arrays
Difference	Elements in first array but not in second array
Membership	Check whether a value belongs to the array/set

Example arrays:

```
A = [2, 6, 10]
B = [3, 6, 7]
```

Results:

```
Union = [2, 3, 6, 7, 10]
Intersection = [6]
A - B = [2, 10]
Membership of 6 in A = true
```

17. Sorted vs Unsorted Set Operations

Set operations can be done on unsorted arrays or sorted arrays.

Unsorted arrays

For unsorted arrays, we may need repeated searching.

Example:

```
A = [10, 2, 6]
B = [7, 6, 3]
```

To find common elements, we may check each element of **A** against many elements of **B**.

This can become slow.

Sorted arrays

For sorted arrays, we can use pointer movement like merge.

Example:

```
A = [2, 6, 10]
B = [3, 6, 7]
```

Because values are sorted, we can move from left to right.

Simple memory:

```
Unsorted set operations often need repeated searching.
Sorted set operations can use merge-style pointer movement.
```

Part C: Union

18. What Union Means

Union means combining all unique elements from both arrays.

Example:

```
A = [2, 6, 10]
B = [3, 6, 7]
```

Union result:

```
[2, 3, 6, 7, 10]
```

Notice:

6 appears in both arrays, but it is copied only once.

Simple idea:

Union = everything, but no duplicate common values.

19. Union Logic for Sorted Arrays

Use two pointers:

```
i = pointer for first array
j = pointer for second array
k = pointer for result array
```

Compare `arr1->A[i]` and `arr2->A[j]`.

Case 1: First value is smaller

```
arr1->A[i] < arr2->A[j]
```

Copy value from first array.

Move:

```
i++
k++
```


Case 2: Second value is smaller

```
arr2->A[j] < arr1->A[i]
```

Copy value from second array.

Move:

```
j++  
k++
```

Case 3: Values are equal

```
arr1->A[i] == arr2->A[j]
```

Copy the value only once.

Move:

```
i++  
j++  
k++
```

Then copy leftovers from either array.

20. Union Dry Run

Given:

```
A = [2, 6, 10]  
B = [3, 6, 7]
```

Start:

```
i = 0, j = 0, k = 0  
C = []
```

Dry run:

Step	Compare	Action	C
1	2 vs 3	Copy 2	[2]
2	6 vs 3	Copy 3	[2, 3]
3	6 vs 6	Copy 6 once	[2, 3, 6]
4	10 vs 7	Copy 7	[2, 3, 6, 7]
5	B finished	Copy leftover 10	[2, 3, 6, 7, 10]

Final union:

[2, 3, 6, 7, 10]

21. Union Code

```

struct Array* Union(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else if (arr2->A[j] < arr1->A[i]) {
            arr3->A[k++] = arr2->A[j++];
        } else {
            arr3->A[k++] = arr1->A[i++];
            j++;
        }
    }

    while (i < arr1->length) {
        arr3->A[k++] = arr1->A[i++];
    }

    while (j < arr2->length) {
        arr3->A[k++] = arr2->A[j++];
    }

    arr3->length = k;
    arr3->size = MAX;
}

```

```
    return arr3;
}
```

22. Explanation of Union Code

Smaller value from first array

```
if (arr1->A[i] < arr2->A[j]) {
    arr3->A[k++] = arr1->A[i++];
}
```

If the first array has the smaller value, copy it.

Smaller value from second array

```
else if (arr2->A[j] < arr1->A[i]) {
    arr3->A[k++] = arr2->A[j++];
}
```

If the second array has the smaller value, copy it.

Equal values

```
else {
    arr3->A[k++] = arr1->A[i++];
    j++;
}
```

If both values are equal, copy only one value.

Then move both pointers.

Why?

Because union should not repeat common elements.

Copy leftovers

```
while (i < arr1->length) {
    arr3->A[k++] = arr1->A[i++];
}
```

```
while (j < arr2->length) {  
    arr3->A[k++] = arr2->A[j++];  
}
```

Union includes all values, so leftover values must be copied.

23. Union Complexity

For sorted arrays:

Complexity type	Result
Time	$O(m + n)$
Space	$O(m + n)$

Why?

Both pointers move forward only once.

For unsorted arrays, union can be slower because we may need to search repeatedly to avoid duplicates.

Common unsorted time:

$O(N^2)$

Part D: Intersection

24. What Intersection Means

Intersection means taking only the values that are present in both arrays.

Example:

A = [2, 6, 10]
B = [3, 6, 7]

Intersection result:

```
[6]
```

Why?

Because `6` is the only value found in both arrays.

Simple idea:

```
Intersection = common values only.
```

25. Intersection Logic for Sorted Arrays

Use two pointers:

```
i for first array  
j for second array  
k for result array
```

Case 1: First value is smaller

```
arr1->A[i] < arr2->A[j]
```

Move `i` forward.

Why?

Because the smaller value cannot match the current larger value in the second array.

Case 2: Second value is smaller

```
arr2->A[j] < arr1->A[i]
```

Move `j` forward.

Case 3: Values are equal

```
arr1->A[i] == arr2->A[j]
```

Copy the value into result.

Move all three:

```
i++  
j++  
k++
```

Important:

Intersection does not copy leftovers.

Why?

If one array has finished, remaining values in the other array cannot be common.

26. Intersection Dry Run

Given:

```
A = [2, 6, 10]  
B = [3, 6, 7]
```

Start:

```
i = 0, j = 0, k = 0  
C = []
```

Dry run:

Step	Compare	Action	C
1	2 vs 3	2 is smaller, move i	[]
2	6 vs 3	3 is smaller, move j	[]
3	6 vs 6	Equal, copy 6	[6]
4	10 vs 7	7 is smaller, move j	[6]
5	B finished	Stop	[6]

Final intersection:

[6]

27. Intersection Code

```
struct Array* Intersection(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            i++;
        } else if (arr2->A[j] < arr1->A[i]) {
            j++;
        } else {
            arr3->A[k++] = arr1->A[i++];
            j++;
        }
    }

    arr3->length = k;
    arr3->size = MAX;

    return arr3;
}
```

28. Explanation of Intersection Code

If first value is smaller

```
if (arr1->A[i] < arr2->A[j]) {
    i++;
}
```

The value from first array is not common, so skip it.

If second value is smaller

```
else if (arr2->A[j] < arr1->A[i]) {  
    j++;  
}
```

The value from second array is not common, so skip it.

If both are equal

```
else {  
    arr3->A[k++] = arr1->A[i++];  
    j++;  
}
```

The value is common, so copy it.

Then move both input pointers.

No leftover loops

Intersection does not need leftover loops.

Reason:

Leftover values cannot be common if the other array is already finished.

29. Intersection Complexity

For sorted arrays:

Complexity type	Result
Time	$O(m + n)$
Space	$O(\min(m, n))$ in ideal capacity thinking, or $O(m + n)$ if result array is allocated with maximum capacity

Why time is $O(m + n)$:

Both pointers move forward and no pointer moves backward.

For unsorted arrays, intersection can be:

$O(N^2)$

because each value from one array may be searched in the other array.

Part E: Difference

30. What Difference Means

Difference means taking elements that are in the first array but not in the second array.

It is usually written as:

A - B

This means:

Values in A that are not in B.

Example:

A = [2, 6, 10]
B = [3, 6, 7]

Difference result:

A - B = [2, 10]

Why?

- 2 is in A but not in B.
- 6 is in both, so skip it.
- 10 is in A but not in B.

Simple idea:

Difference = keep only values unique to the first array.

31. Difference is Directional

This is very important.

A - B is not always the same as B - A.

Example:

A = [2, 6, 10]
B = [3, 6, 7]

Then:

A - B = [2, 10]
B - A = [3, 7]

So difference depends on which array comes first.

Simple memory:

A - B means keep A's values that are not in B.
B - A means keep B's values that are not in A.

32. Difference Logic for Sorted Arrays

Use two pointers:

i for first array
j for second array
k for result array

Case 1: A[i] is smaller

```
arr1->A[i] < arr2->A[j]
```

Copy `arr1->A[i]`.

Why?

Because this value exists in the first array and is smaller than the current value in the second array, so it cannot be present at `arr2[j]` or later.

Move:

```
i++  
k++
```

Case 2: B[j] is smaller

```
arr2->A[j] < arr1->A[i]
```

Move `j` forward.

Why?

Because this value belongs only to the second array, and `A - B` does not copy values from B.

Case 3: Values are equal

```
arr1->A[i] == arr2->A[j]
```

Skip both.

Why?

Because common values should not appear in `A - B`.

Move:

```
i++  
j++
```

After the main loop, copy leftover values only from the first array.

33. Difference Dry Run

Given:

```
A = [2, 6, 10]
B = [3, 6, 7]
```

Find:

```
A - B
```

Start:

```
i = 0, j = 0, k = 0
C = []
```

Dry run:

Step	Compare	Action	C
1	2 vs 3	2 is smaller, copy 2	[2]
2	6 vs 3	3 is smaller, move j	[2]
3	6 vs 6	Equal, skip both	[2]
4	10 vs 7	7 is smaller, move j	[2]
5	B finished	Copy leftover 10 from A	[2, 10]

Final result:

```
A - B = [2, 10]
```

34. Difference Code

```
struct Array* Difference(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
```

```

    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else if (arr2->A[j] < arr1->A[i]) {
            j++;
        } else {
            i++;
            j++;
        }
    }

    while (i < arr1->length) {
        arr3->A[k++] = arr1->A[i++];
    }

    arr3->length = k;
    arr3->size = MAX;

    return arr3;
}

```

35. Explanation of Difference Code

Copy from first array when smaller

```

if (arr1->A[i] < arr2->A[j]) {
    arr3->A[k++] = arr1->A[i++];
}

```

This value is in first array and not found in second array at this position.

So copy it.

Move second pointer when second value is smaller

```

else if (arr2->A[j] < arr1->A[i]) {
    j++;
}

```

This value belongs to second array.

Since difference is `A - B`, we do not copy values from B.

Skip equal values

```
else {  
    i++;  
    j++;  
}
```

If values are equal, they are common.

Common values are not included in `A - B`.

Copy leftovers from first array only

```
while (i < arr1->length) {  
    arr3->A[k++] = arr1->A[i++];  
}
```

Why only from first array?

Because `A - B` only keeps values from A.

Leftover values from B are ignored.

36. Difference Complexity

For sorted arrays:

Complexity type	Result
Time	<code>O(m + n)</code>
Space	Up to <code>O(m)</code> for actual result values, or <code>O(m + n)</code> if using a large fixed result capacity

For unsorted arrays:

```
O(N2)
```

Reason:

Each value from the first array may need to be searched in the second array.

Part F: Membership

37. What Membership Means

Membership means checking whether a value exists in a set or array.

Question:

Is key present in the array?

Example:

A = [2, 6, 10]
key = 6

Answer:

Yes, 6 is present.

If the key is not found:

No, key is not present.

38. Membership in Unsorted Arrays

If the array is unsorted, use linear search.

Example:

A = [10, 2, 6]
key = 6

We may need to check values one by one:

```
10 -> not found
2 -> not found
6 -> found
```

Time complexity:

```
O(N)
```

39. Membership in Sorted Arrays

If the array is sorted, use binary search.

Example:

```
A = [2, 6, 10, 15, 25]
key = 10
```

Binary search can check the middle and reduce the search area quickly.

Time complexity:

```
O(log N)
```

Simple memory:

```
Unsorted membership = linear search = O(N)
Sorted membership = binary search = O(log N)
```

Part G: Full Program for Phase 8

40. Complete C Program

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 50
```



```

struct Array {
    int A[MAX];
    int size;
    int length;
};

void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}

struct Array* merge(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else {
            arr3->A[k++] = arr2->A[j++];
        }
    }

    while (i < arr1->length) {
        arr3->A[k++] = arr1->A[i++];
    }

    while (j < arr2->length) {
        arr3->A[k++] = arr2->A[j++];
    }

    arr3->length = k;
    arr3->size = MAX;
    return arr3;
}

struct Array* Union(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else if (arr2->A[j] < arr1->A[i]) {
            arr3->A[k++] = arr2->A[j++];
        }
    }

```

```

        } else {
            arr3->A[k++] = arr1->A[i++];
            j++;
        }
    }

    while (i < arr1->length) {
        arr3->A[k++] = arr1->A[i++];
    }

    while (j < arr2->length) {
        arr3->A[k++] = arr2->A[j++];
    }

    arr3->length = k;
    arr3->size = MAX;
    return arr3;
}

struct Array* Intersection(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            i++;
        } else if (arr2->A[j] < arr1->A[i]) {
            j++;
        } else {
            arr3->A[k++] = arr1->A[i++];
            j++;
        }
    }

    arr3->length = k;
    arr3->size = MAX;
    return arr3;
}

struct Array* Difference(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else if (arr2->A[j] < arr1->A[i]) {
            j++;
        }
    }

```

```

        } else {
            i++;
            j++;
        }
    }

    while (i < arr1->length) {
        arr3->A[k++] = arr1->A[i++];
    }

    arr3->length = k;
    arr3->size = MAX;
    return arr3;
}

int binarySearch(struct Array arr, int key) {
    int low = 0;
    int high = arr.length - 1;
    int mid;

    while (low <= high) {
        mid = (low + high) / 2;

        if (key == arr.A[mid]) {
            return mid;
        } else if (key < arr.A[mid]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    return -1;
}

int main() {
    struct Array arr1 = {{2, 6, 10, 15, 25}, MAX, 5};
    struct Array arr2 = {{3, 6, 7, 15, 20}, MAX, 5};

    struct Array *result;

    printf("Array 1:\n");
    display(arr1);

    printf("Array 2:\n");
    display(arr2);

    result = merge(&arr1, &arr2);

```

```

    printf("Merged array:\n");
    display(*result);
    free(result);

    result = Union(&arr1, &arr2);
    printf("Union:\n");
    display(*result);
    free(result);

    result = Intersection(&arr1, &arr2);
    printf("Intersection:\n");
    display(*result);
    free(result);

    result = Difference(&arr1, &arr2);
    printf("Difference A - B:\n");
    display(*result);
    free(result);

    int key = 10;
    if (binarySearch(arr1, key) != -1) {
        printf("%d is present in Array 1.\n", key);
    } else {
        printf("%d is not present in Array 1.\n", key);
    }

    return 0;
}

```

41. Program Walkthrough

Starting arrays:

```

arr1 = [2, 6, 10, 15, 25]
arr2 = [3, 6, 7, 15, 20]

```

Merge result

Merge keeps all values from both arrays in sorted order.

```
[2, 3, 6, 6, 7, 10, 15, 15, 20, 25]
```

Notice:

Merge does not remove duplicates.

Union result

Union keeps all unique values.

[2, 3, 6, 7, 10, 15, 20, 25]

Common values like 6 and 15 appear only once.

Intersection result

Intersection keeps only common values.

[6, 15]

Difference result

Difference `arr1 - arr2` keeps values in `arr1` that are not in `arr2`.

[2, 10, 25]

Membership result

For key 10 in `arr1`:

10 is present in Array 1.

Part H: Merge vs Set Operations

42. Merge vs Union

Merge and union look similar, but they are not the same.

Operation	Copies duplicates?	Example result
Merge	Yes	[2, 3, 6, 6, 7]

Operation	Copies duplicates?	Example result
Union	No	[2, 3, 6, 7]

Example:

A = [2, 6]
B = [3, 6]

Merge:

[2, 3, 6, 6]

Union:

[2, 3, 6]

Simple memory:

Merge combines sorted arrays.
Union combines set values without repeating common elements.

43. Union vs Intersection vs Difference

Given:

A = [2, 6, 10]
B = [3, 6, 7]

Operation	Meaning	Result
Union	Everything unique from both arrays	[2, 3, 6, 7, 10]
Intersection	Common values only	[6]
Difference A - B	Values in A but not in B	[2, 10]
Difference B - A	Values in B but not in A	[3, 7]

Simple memory:

Union = all unique values.
Intersection = common values.
Difference = values only in the first array.

Part I: Complexity Summary

44. Complexity of Sorted Operations

Let:

m = length of first array
n = length of second array

Operation	Time Complexity	Why
Merge	$O(m + n)$	Copies all elements once
Union	$O(m + n)$	Moves through both arrays once
Intersection	$O(m + n)$	Moves through both arrays once
Difference	$O(m + n)$	Moves through both arrays once
Membership using binary search	$O(\log N)$	Search area halves each step

45. Complexity of Unsorted Operations

For unsorted arrays:

Operation	Common Time Complexity	Why
Union	$O(N^2)$	May need repeated duplicate checking
Intersection	$O(N^2)$	May search one array for every value of the other
Difference	$O(N^2)$	May search second array for every value in first
Membership using linear search	$O(N)$	Checks values one by one

Important idea:

Sorted arrays make set operations faster.

Why?

Because sorted arrays allow pointer movement instead of repeated searching.

46. Space Complexity Summary

Operation		Extra Space
Merge		$O(m + n)$
Union		Up to $O(m + n)$
Intersection	Up to $O(\min(m, n))$, or fixed result capacity in simple C programs	
Difference		Up to $O(m)$
Membership		$O(1)$ for iterative search

Beginner note:

In many beginner C programs, we create a result array with a fixed maximum capacity, such as `MAX = 50`.

In algorithm analysis, we usually describe space based on how many result values may be stored.

Part J: Important Implementation Details

47. Why Result Length Must Be `k`

In merge and set operations, `k` counts how many values were actually copied into the result array.

So we must write:

```
arr3->length = k;
```

Do not always write:


```
arr3->length = arr1->length + arr2->length;
```

Why?

Because set operations may skip values.

Example:

```
A = [2, 6, 10]
B = [3, 6, 7]
```

For intersection:

```
Result = [6]
```

Here:

```
k = 1
arr1.length + arr2.length = 6
```

If we set length to `6`, display will print invalid extra values.

Simple memory:

```
The result length is the number of values actually copied.
That number is k.
```

48. Why Equal Values Are Handled Differently

Equal values are handled differently in different operations.

Given:

```
A[i] == B[j]
```

Operation	What to do with equal values?	Why
Merge	Copy both, depending on implementation flow	Merge keeps duplicates

Operation	What to do with equal values?	Why
Union	Copy once	Union avoids duplicates
Intersection	Copy once	Equal means common value
Difference	Skip both	Common values are not in A - B

Simple memory:

```
Equal value in union: copy once.
Equal value in intersection: copy once.
Equal value in difference: skip.
```

49. Why Intersection Has No Leftover Loop

Intersection means common values.

If one array is finished, no more common values can be found.

Example:

```
A = [2, 4, 6]
B = [2]
```

After **B** is finished, the leftover values in **A** are:

```
4, 6
```

They cannot be common because **B** has no values left.

So intersection does not copy leftovers.

50. Why Difference Copies Leftovers Only From First Array

Difference **A - B** means:

```
Values in A that are not in B.
```

So only values from A can appear in the result.

If B finishes first, leftover A values must be copied.

If A finishes first, leftover B values are ignored.

Simple memory:

Difference only copies from the first array.

51. Why We Use `free()`

When we create memory using `malloc`, we should release it using `free`.

Example:

```
struct Array *result = merge(&arr1, &arr2);
display(*result);
free(result);
```

Why?

Because `malloc` creates heap memory.

If we do not release heap memory, larger programs may waste memory.

This is called a memory leak.

Simple memory:

`malloc` needs `free`.

Part K: Common Beginner Mistakes

52. Mistake 1: Merging Unsorted Arrays

Wrong idea:

```
A = [10, 2, 6]
B = [7, 3, 9]
```

Trying to merge these directly will not produce a sorted result.

Correct idea:

```
Merge requires sorted input arrays.
```

53. Mistake 2: Using `size` Instead of `length`

Wrong:

```
while (i < arr1->size && j < arr2->size)
```

Correct:

```
while (i < arr1->length && j < arr2->length)
```

Why?

`size` includes unused capacity.

`length` tells how many real values exist.

54. Mistake 3: Setting Result Length Incorrectly

Wrong:

```
arr3->length = arr1->length + arr2->length;
```

This may work for merge, but not for union, intersection, or difference.

Correct:

```
arr3->length = k;
```

Why?

`k` tells how many values were actually copied.

55. Mistake 4: Copying Equal Values Twice in Union

Wrong union result:

```
A = [2, 6]
B = [3, 6]
Union = [2, 3, 6, 6]
```

Correct union result:

```
[2, 3, 6]
```

If values are equal, copy once and move both pointers.

56. Mistake 5: Copying Leftovers in Intersection

Wrong idea:

```
After one array ends, copy the rest.
```

This is wrong for intersection.

Correct idea:

```
Intersection only contains common values.
No leftover values should be copied.
```

57. Mistake 6: Returning Address of a Local Variable

Wrong:

```
struct Array* merge(struct Array *arr1, struct Array *arr2) {  
    struct Array arr3;  
    return &arr3;  
}
```

Why wrong?

`arr3` is local to the function.

It disappears after the function ends.

Correct:

```
struct Array *arr3 = malloc(sizeof(struct Array));  
return arr3;
```

Or create the result array in `main` and pass it to the function.

58. Mistake 7: Forgetting to Free Result Arrays

Wrong:

```
struct Array *result = merge(&arr1, &arr2);  
display(*result);
```

Correct:

```
struct Array *result = merge(&arr1, &arr2);  
display(*result);  
free(result);
```

If you use `malloc`, remember to use `free` later.

Part L: Quick Revision Tables

59. Operation Meaning Table

Operation	Meaning
Merge	Combine two sorted arrays into one sorted array
Union	Combine all unique elements from both arrays
Intersection	Keep only values common to both arrays
Difference $A - B$	Keep values in A that are not in B
Membership	Check whether a value is present

60. Pointer Action Table

Situation	Merge	Union	Intersection	Difference $A - B$
$A[i] < B[j]$	Copy $A[i]$	Copy $A[i]$	Move i	Copy $A[i]$
$B[j] < A[i]$	Copy $B[j]$	Copy $B[j]$	Move j	Move j
$A[i] == B[j]$	Copy one or both depending on duplicate handling	Copy once	Copy once	Skip both
Leftover A	Copy	Copy	Ignore	Copy
Leftover B	Copy	Copy	Ignore	Ignore

61. Complexity Table

Operation	Sorted Time	Unsorted Time
Merge	$O(m + n)$	Not directly suitable unless arrays are sorted first
Union	$O(m + n)$	$O(N^2)$
Intersection	$O(m + n)$	$O(N^2)$
Difference	$O(m + n)$	$O(N^2)$

Operation	Sorted Time	Unsorted Time
Membership	$O(\log N)$ with binary search	$O(N)$ with linear search

Part M: Checkpoint Questions

62. Concept Checkpoints

You are ready to move on from Phase 8 when you can answer these without looking:

1. What does a binary operation mean in Array ADT?
2. Why must arrays be sorted before using merge?
3. What do `i`, `j`, and `k` represent in merge?
4. Why does merge need leftover loops?
5. What is the difference between merge and union?
6. Why does union copy equal values only once?
7. Why does intersection not copy leftovers?
8. Why does difference copy leftovers only from the first array?
9. Why should result length be `k`?
10. Why is sorted union $O(m + n)$?
11. Why is membership faster in a sorted array?
12. Why is returning a pointer to a local variable unsafe?
13. Why should `free()` be used after `malloc()`?

63. Practice Problems

Problem 1: Merge

Given:

```
A = [1, 5, 9]
B = [2, 3, 10]
```

Find the merged array.

Answer:

```
[1, 2, 3, 5, 9, 10]
```


Problem 2: Union

Given:

A = [1, 3, 5, 7]
B = [3, 4, 5, 8]

Find the union.

Answer:

[1, 3, 4, 5, 7, 8]

Problem 3: Intersection

Given:

A = [1, 3, 5, 7]
B = [3, 4, 5, 8]

Find the intersection.

Answer:

[3, 5]

Problem 4: Difference

Given:

A = [1, 3, 5, 7]
B = [3, 4, 5, 8]

Find $A - B$.

Answer:

[1, 7]

Problem 5: Difference Direction

Using the same arrays:

A = [1, 3, 5, 7]
B = [3, 4, 5, 8]

Find `B - A`.

Answer:

[4, 8]

Important:

A - B and B - A are different.

64. Final Phase 8 Summary

Phase 8 teaches operations that use two arrays.

The most important operation is merge.

Merge combines two sorted arrays into one sorted result.

Set operations are based on similar pointer movement:

Union = all unique values.
Intersection = common values only.
Difference = values in the first array but not in the second.
Membership = checking whether a value exists.

The biggest advantage of sorted arrays is speed.

For sorted arrays:

Union, intersection, and difference can run in $O(m + n)$.

For unsorted arrays, these operations are often slower because they may require repeated searching.

Final memory:

Phase 8 = two arrays, three pointers, one result array.

Merge copies everything in sorted order.

Union copies all unique values.

Intersection copies only common values.

Difference copies only values from the first array.

Membership checks whether a value exists.